IP PROJECT REPORT

SkillSwap

A Skill Exchange Platform

PRESENTED BY: TT-AI&ML-C
Semester V

Tiya Rai [03]

Academic Year 2026-27

1. Abstract

SkillSwap is a website where people exchange skills instead of paying for lessons. A person lists the skills they can teach and the skills they want to learn, and the system finds people whose lists fit together. Two people who can each teach the other something become a mutual match and can start a swap. The server is built with Spring Boot, the pages with React, and all the data is saved in a MySQL database.

When a swap is accepted, the two people get a chat, a video call link and a learning period of the days they agreed on. At the end each person writes a multiple choice test for the other. The application marks the test by itself, and when both people score 50 percent or more, the swap is completed and they can rate each other.

2. Introduction

Many people know something well and want to learn something else, but paying for classes is not always possible. SkillSwap lets them trade what they know. Someone who knows Python can teach it to a person who plays the guitar, and the guitar player can teach back. No money is involved. To register, a person needs a Gmail address, a date of birth and a strong password, and a forgotten password can be reset with the date of birth.

Objectives

- To let people list the skills they can teach and the skills they want to learn.
- To find matches automatically and show the mutual matches first.
- To let two people send, accept, reject or cancel a swap and agree on the number of days.
- To give every accepted swap a chat and a video call link.
- To check what was learned with a multiple choice test that the application marks.
- To save all data in a real MySQL database and keep passwords safe.

3. Project Description

3.1 System Architecture

SkillSwap has two programs that talk to each other. The React website runs in the browser and sends requests to the Spring Boot server. The server is split into simple layers, and every error is sent back in the same format so the website can show it under the right field.

Layer	Files and Responsibility
Controller	REST controllers that receive a request and send back the answer, one for each area
Service	The rules: matching, swap rules, tests and marking, password reset and account deletion
Repository	Spring Data JPA interfaces that read and write the MySQL tables
Entity	Java classes for the tables: User, Skill, UserSkill, SwapRequest, SwapMessage, Assessment and Review
Security	JwtUtil, JwtAuthFilter and SecurityConfig: login tokens, password hashing and admin only actions
Website	React pages, reusable parts such as cards, forms, chat and test screens, and the API files

3.2 Key Components Used

- **Spring Boot, Spring Data JPA and Hibernate:** run the server and create the MySQL tables on the first start.
- **Spring Security, JWT and BCrypt:** check the login token on every request and save passwords as hashes.
- **MySQL and Connector/J:** the real database and its driver.
- **React, Vite and Tailwind CSS:** build the pages in the light purple and white design.
- **Swagger UI, JUnit and Mockito:** try every server call and run 92 automatic tests.
- **Ollama and Jitsi Meet:** a small AI model on the same computer drafts test questions, and Jitsi gives a free call room.

3.3 Module Descriptions

Registration and Login. A person registers with a name, a Gmail address, a date of birth and a password with six or more characters, an uppercase letter, a lowercase letter, a digit and a special character. A forgotten password is reset with the Gmail address and the date of birth. The login is kept per browser tab, so two people can be logged in on two tabs.

Profile. A person can change the name, bio, date of birth and a ten digit Indian mobile number, choose a profile photo from the computer, and delete the account after entering the password again.

Skills and Matching. People add skills from a shared list to what they can teach or want to learn, and can create a new skill if it is missing. If another person teaches what I want to learn and wants what I teach, it is a mutual match. If only one side fits, it is a one way match.

Swaps and Workspace. From a match a person sends a request with the skills, the number of days and a message. After it is accepted, the two people get a learning period, a chat that refreshes every three seconds and a Jitsi call link that either person can replace with their own link.

Final Tests. When the learning period ends, each person writes a multiple choice test for the other, or asks the local AI model for a draft from a topic and then edits it. The application marks the answers at once, gives points and says whether the person passed. The pass mark is 50 percent, and a failed test can be taken again.

Reviews and Admin. After a completed swap each person gives one to five stars. The admin can add, change and delete skills in the shared list.

3.4 Database Schema

The server creates the MySQL database skillswap and its eight tables on the first start:

Table	Primary Key	Key Fields
users	id	name, email, password (hash), date_of_birth, phone, avatar_url, role
skills	id	name, category, description
user_skills	id	user_id, skill_id, type (teach or learn), level
swap_requests	id	requester_id, receiver_id, skills, status, duration_days, start_date, end_date
swap_messages	id	swap_request_id, sender_id, text, created_at
assessments	id	swap_request_id, teacher_id, learner_id, status, score_percent, passed

Table	Primary Key	Key Fields
assessment_questions	id	assessment_id, text, options_json, correct_option, selected_option
reviews	id	swap_request_id, reviewer_id, reviewee_id, rating, comment

3.5 Software and Hardware Requirements

Software	Hardware
Java (JDK 21 or newer), Spring Boot 3.5 React with Vite (Node.js 20 or newer) MySQL 8.0 or newer, Connector/J 9.6.0 VS Code or IntelliJ IDEA, Postman Optional: Ollama with the phi3 model	Processor: Intel i3 or above RAM: Minimum 8 GB (16 GB with Ollama) Storage: 2 GB free space Display: 1280x768 minimum resolution Internet: needed only for the video call

4. Results and Discussion

SkillSwap was built and tested against the planned features. A person can register, log in, add skills, find matches, send a swap, chat, take a test and finish a swap from start to end. The project has 92 automatic tests, including one that follows a whole swap from the request to completion with a failed test and a retake. All of them pass.

Passwords are saved as hashes, and every request except login, register and password reset needs a valid token. Only the admin can edit or delete shared skills, and only the two people of a swap can read its chat and tests. Every form shows problems under the field, and each save shows a green success message.

The AI question writer needs about one minute for five questions on a computer without a graphics card. A small model can make mistakes, so the teacher checks the questions and the right answer before sending.

The pictures below are taken from the running project. Figure 1 is the landing page. Figure 2 is the dashboard with the active swaps, the mutual matches and the latest swap activity. Figure 3 is the list of skills that can be searched and added, and Figure 4 shows a person's own skills to teach and to learn. Figure 5 shows the matches with the fitting skills of both sides, and Figure 6 shows the swaps with their status and the buttons to act on them.

OUTPUT:

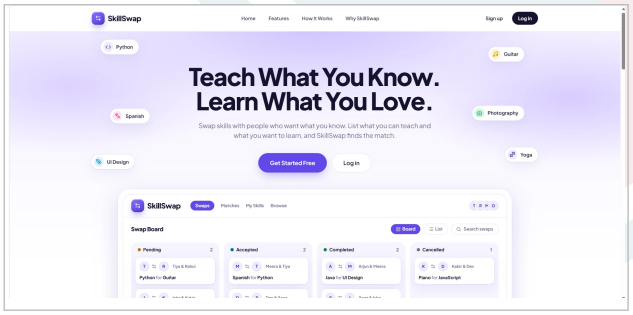


Figure 1: Landing Page

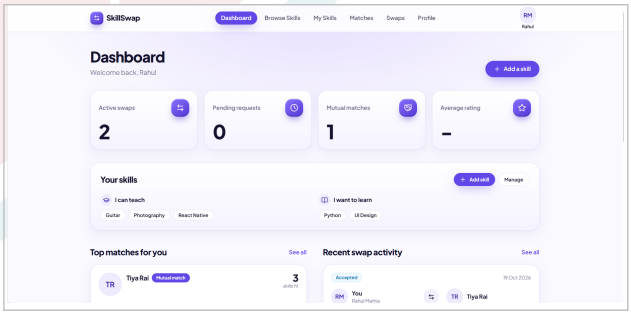


Figure 2: Dashboard

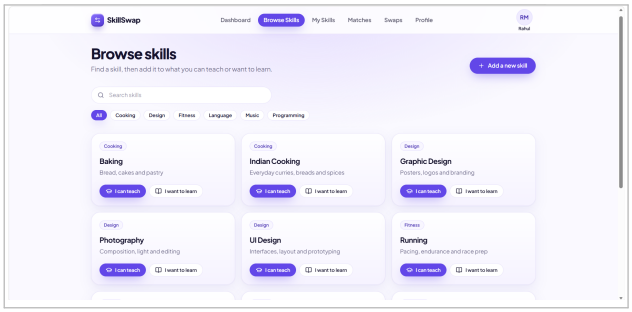


Figure 3: Browse Skills

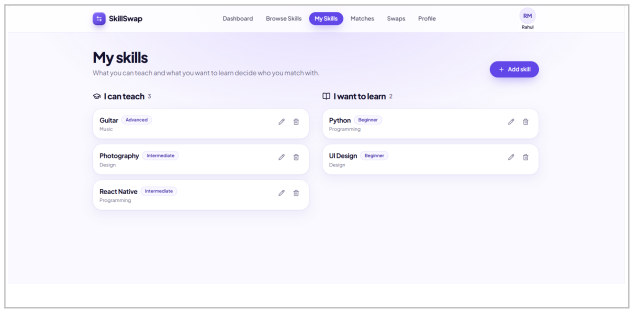


Figure 4: My Skills

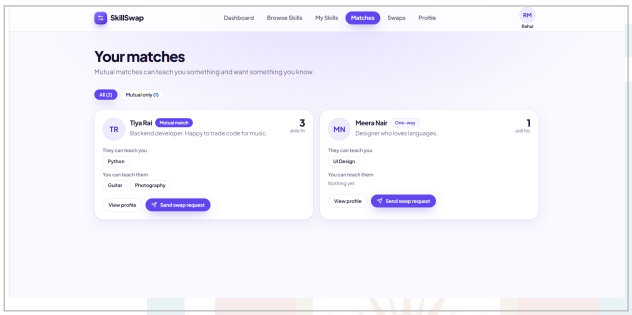


Figure 5: Matches

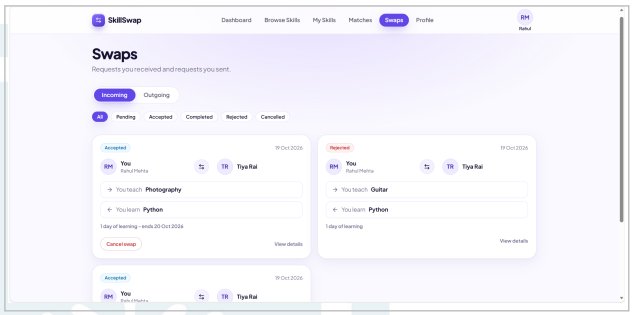


Figure 6: Swaps

5. Conclusion and Future Scope

Conclusion

SkillSwap shows that people can learn from each other without paying. It joins a Spring Boot server, a React website and a MySQL database into one working system, from registration to a completed and reviewed swap. The project gave practice in building web services, saving data with JPA, protecting a website with tokens and writing automatic tests.

Future Scope

- Send a password reset link by email instead of asking for the date of birth.
- Add a video call inside the website and send notifications for new requests and messages.
- Give a certificate when a swap is completed and show the admin simple charts.
- Make a mobile app for Android and iOS.
- Add other question types, such as short written answers, and explain on each match why two people fit.

6. References

- Spring Boot and Spring Security Reference Documentation, spring.io
- MySQL 8.0 Reference Manual and Connector/J Documentation, dev.mysql.com
- React Documentation, react.dev, and Tailwind CSS Documentation, tailwindcss.com
- JSON Web Token Introduction, jwt.io
- Ollama Documentation, ollama.com, and Jitsi Meet, jitsi.org
- Hibernate ORM Documentation, hibernate.org
- Baeldung Spring Tutorials, baeldung.com
- GeeksforGeeks Spring Boot Guide, geeksforgeeks.org

7. Annexure (Source Code)

A few short parts of the source code are shown below. The complete project is in the backend and frontend folders.

User.java (the users table)

```
// JPA entity: maps to the "users" table
@Entity
@Table(name = "users")
@Getter
@Setter
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    // BCrypt hash, never the plain password
    @Column(nullable = false)
    private String password;

    @Column(length = 500)
    private String bio;

    // path of the uploaded profile photo, e.g. /api/files/avatars/abc.png
    private String avatarUrl;

    // used to verify the person when they reset a forgotten password
    private LocalDate dateOfBirth;

    // normalized Indian mobile number, e.g. +919876543210
    @Column(length = 15)
    private String phone;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;

    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() {
        createdAt = LocalDateTime.now();
    }
}
```

SwapRequestController.java (swap requests)

```
@RestController
@RequestMapping("/api/swaps")
@Tag(name = "Swap requests")
public class SwapRequestController {

    private final SwapService swapService;

    public SwapRequestController(SwapService swapService) {
        this.swapService = swapService;
```

```
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @Operation(summary = "Send a swap request")
    @ApiResponse(responseCode = "409", description = "A pending request for this swap already exists")
    public SwapResponse create(Principal principal, @Valid @RequestBody SwapCreateRequest request) {
        return swapService.create(principal.getName(), request);
    }

    @GetMapping
    @Operation(summary = "My incoming or outgoing requests (paginated)")
    public PageResponse<SwapResponse> list(Principal principal,
                                           @RequestParam(defaultValue = "incoming") String box,
                                           @RequestParam(required = false) SwapStatus status,
                                           @RequestParam(defaultValue = "0") int page,
                                           @RequestParam(defaultValue = "10") int size) {
        return swapService.list(principal.getName(), box, status, page, size);
    }

    @GetMapping("/{id}")
    @Operation(summary = "One request (only if I am a participant)")
    public SwapResponse get(Principal principal, @PathVariable Long id) {
        return swapService.get(principal.getName(), id);
    }

    @PatchMapping("/{id}/status")
    @Operation(summary = "Accept, reject, cancel or complete a request")
    @ApiResponse(responseCode = "409", description = "That status change is not allowed right now")
    public SwapResponse updateStatus(Principal principal, @PathVariable Long id,
                                     @Valid @RequestBody SwapStatusRequest request) {
        return swapService.updateStatus(principal.getName(), id, request.status());
    }
}
```

AssessmentService.java (marking a test)

```
        int total = 0;
        int earned = 0;
        for (AssessmentQuestion q : assessment.getQuestions()) {
            Integer picked = picks.get(q.getId());
            int choices = readOptions(q).size();
            if (picked == null || picked < 0 || picked >= choices) {
                throw new BadRequestException("Please pick an answer for every question");
            }
            q.setSelectedOption(picked);
            boolean correct = picked.equals(q.getCorrectOption());
            q.setAwardedPoints(correct ? q.getMaxPoints() : 0);
            total += q.getMaxPoints();
            earned += q.getAwardedPoints();
        }

        int percent = total == 0 ? 0 : (int) Math.round(earned * 100.0 / total);
        assessment.setScorePercent(percent);
        assessment.setPassed(percent >= PASS_PERCENT);
```

SwapRequest.java (when the tests open)

```
    /** Assessments open when the learning period is over, or when both people finished early. */
    public boolean isAssessmentUnlocked() {
        if (status != SwapStatus.ACCEPTED || endDate == null) {
            return false;
        }
        boolean periodOver = !LocalDate.now().isBefore(endDate);
        return periodOver || (requesterFinishedEarly && receiverFinishedEarly);
    }
```